

레포 인사이트

GitHub 레포 분석 → Notion 포트폴리오 자동화 도구

성명



1. 프로젝트 개요

1-1. 주제 확정 — 레포 인사이드

누가: 자신이 만든 GitHub 공개 레포를 이력서·포트폴리오에 정리해야 하는 개발자가

무엇을: 레포 URL 하나만 입력하면 구조·기술스택·커밋 히스토리를 자동 분석해 리포트를 만들고, 확인 후 버튼 클릭으로 Notion 포트폴리오 페이지에 업로드하는 도구

왜: 레포를 직접 열어 구조·README·커밋·언어 비율을 눈으로 훑고 손으로 정리하는 반복 작업을 줄이기 위해 만든다.

- 데이터 소스: (b) 공개 데이터 수집 — GitHub REST API(메타데이터·구조·커밋·언어·README)
- 산출물 형태: 리포트 문서(Markdown) + 화면(입력/실행/미리보기/이력) 둘 다
- 민감 데이터 아님 — 커밋 작성자 이메일 등은 마스킹 처리

20~30분 → 5분

레포 1건당 정리 소요시간 절감 목표

약 80%+

시간 절감률 가설

5~10건

이직·이력서 갱신 시기당 정리 레포 수



2. 개발 환경 및 요구사항 분석

2-1. 환경 점검

항목	기대값	실제값 / 조치	결과
Node.js	설치됨	v24.19.0 (npm 11.17.0)	O
Docker Desktop	실행 중	엔진 중지 → 사용자 승인 후 기동 확인	O
k3d	설치·정상	v5.9.0, 기존 클러스터 k3d-myapp-dev 재기동 확인	O
MCP 4종	등록됨	filesystem·context7·Sequential Thinking·Playwright 신규 등록	O
xlsx 생성 수단	가능	Claude Code 내장 xlsx Skill 사용(설치 불필요)	O

- 종합 판단: 최종 x(미해결) 항목 없음 — 세션 시작 시 발견된 두 이슈(Docker 엔진 중지, MCP 미등록)는 사용자 승인 후 모두 해결
- 1차시 산출물(F4_Database_선택가이드.md 등) 재활용 가능성 확인 — 최종 재활용 여부는 주제 확정 후 판단

2-2. 앱기획 스펙 (1/2) — 사용자 스토리 · MVP

#	사용자 스토리	KPI
US1	GitHub 레포 URL 입력 → 구조·기술스택·커밋 요약 리포트 자동 수신	분석 1건: 20~30분 → 5분 이내
US2	생성된 리포트를 별도 복사 없이 Notion에 바로 저장	업로드 원클릭, 수동 복사 0회
US3	과거 분석 이력을 재조회해 포트폴리오 갱신 시 재사용	이력 재조회 100% 가능

필수(MVP): URL 입력→분석→미리보기→Notion 업로드 · SQLite 영속 저장 · 입력검증 · 감사로그(누가·언제·무엇) · 개인정보 마스킹 · 역할 스위치 (관리자/일반, 서버 측 403 차단)

나중(Post-MVP): 다중 사용자 인증(OAuth) · 정기 배치 재분석 · 레포 간 비교 · Slack 연동

2-2. 앱기획 스펙 (2/2) — 인수조건 · 오픈소스 후보

AC	구분	내용
AC1	정상	유효 레포 URL → 분석 실행 → 리포트 화면 표시
AC2	정상	리포트 확인 후 Notion 업로드 → 페이지 링크 표시
AC3	정상	이력 목록에서 과거 분석 재조회
AC4	예외	비공개/미존재 레포 → 안내 후 리포트 미생성
AC5	예외	GitHub Rate Limit 초과 → 재시도 가능 시각 안내
AC6	예외	Notion 토큰 미설정/만료 → 오류 안내, 토큰값 비노출

GitHub 연동: @octokit/rest (MIT)

Notion 연동: @notionhq/client (MIT)

서버: Express (MIT)

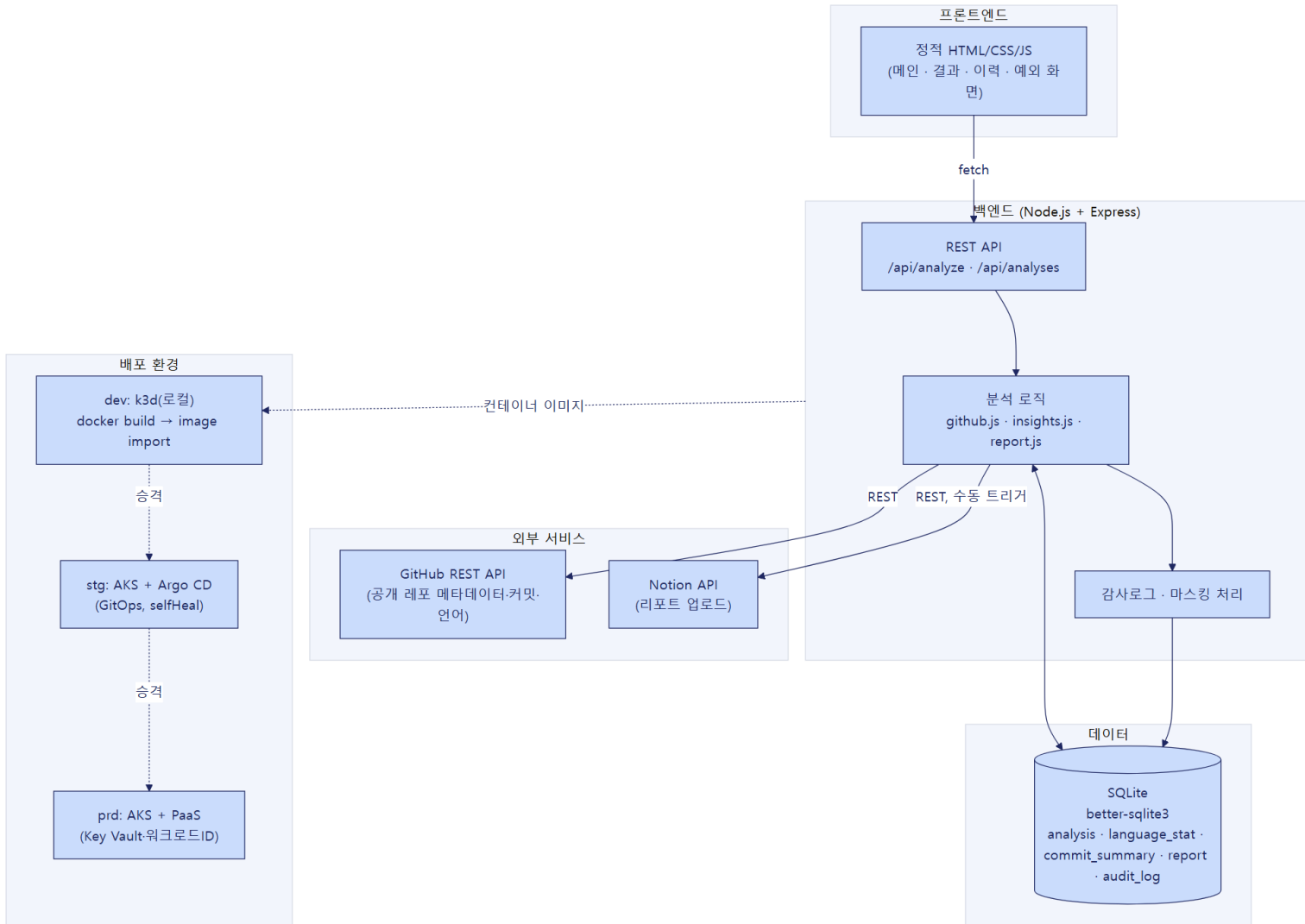
저장소: better-sqlite3 (MIT)

리포트 생성: Markdown 템플릿 문자열(별도 변환 불필요)



3. 앱 설계

3-1. 전체 아키텍처 (프론트엔드 / DB / 백엔드 / 배포)



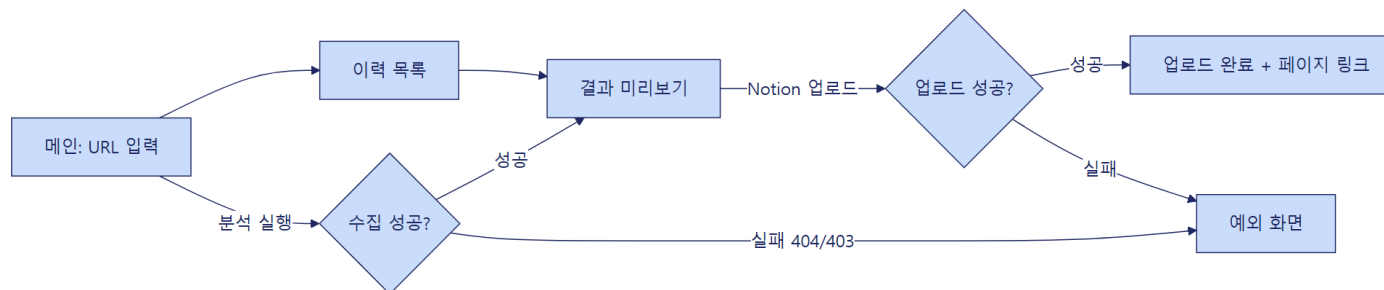
3-2. 화면 설계 (1/2) — 화면 목록 · 흐름도

메인(분석 실행): 레포 URL 입력 → 분석 실행

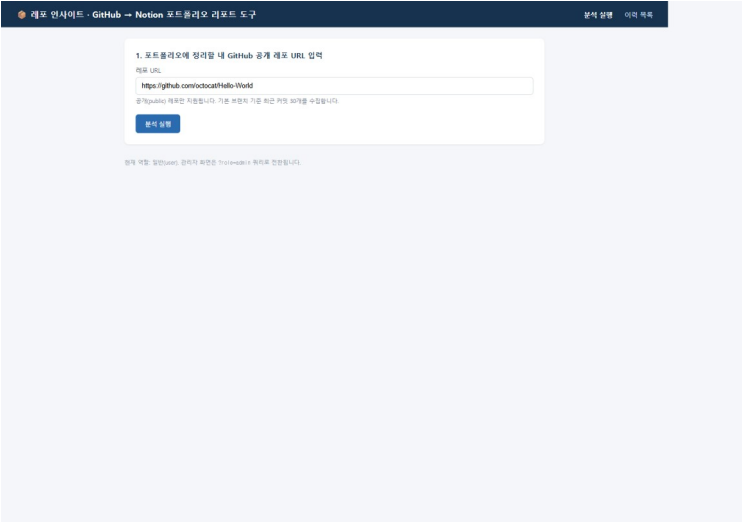
결과 미리보기: 프로젝트 주제·구조·아키텍처 추정.
기술스택 구성비(막대그래프)·커밋 요약, Notion
업로드 버튼

이력 목록: 과거 분석 이력 테이블, 상세 재조회

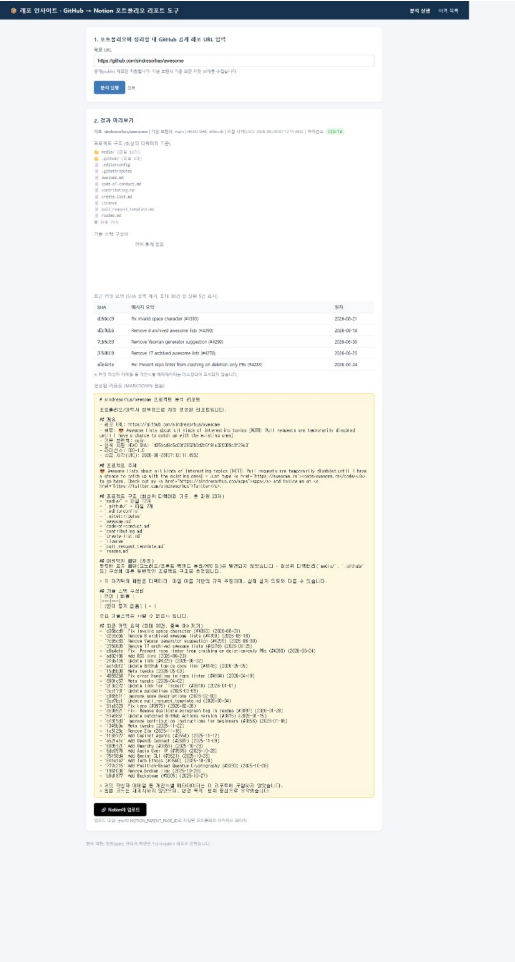
예외 화면: 비공개/존재하지 않는 레포, Rate Limit 초과, Notion 업로드 실패 안내



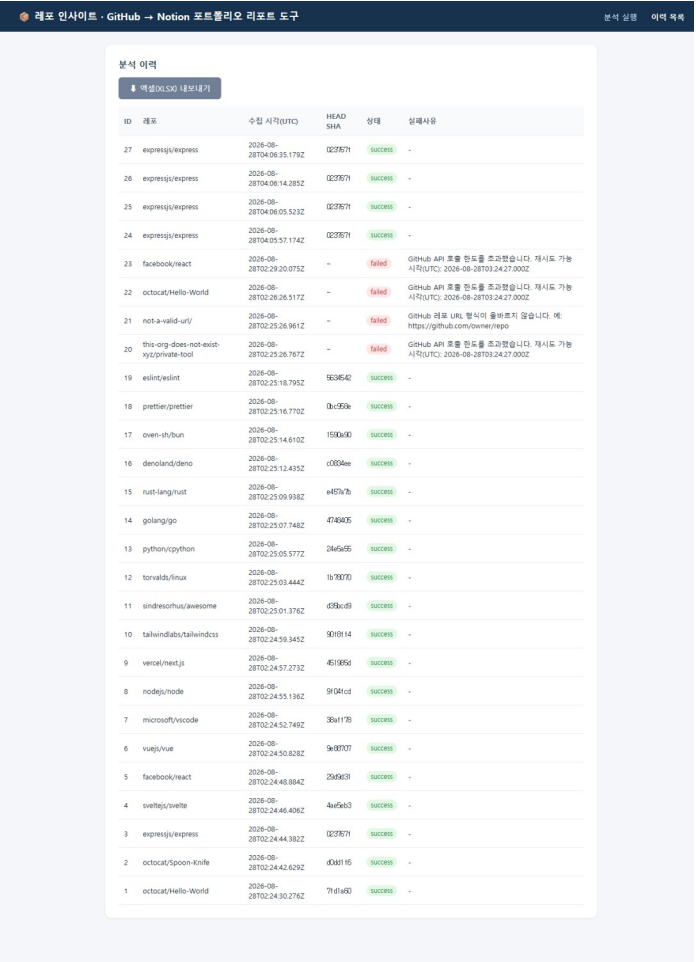
3-2. 화면 설계 (2/2) — 실제 실행 화면



① 초기 화면

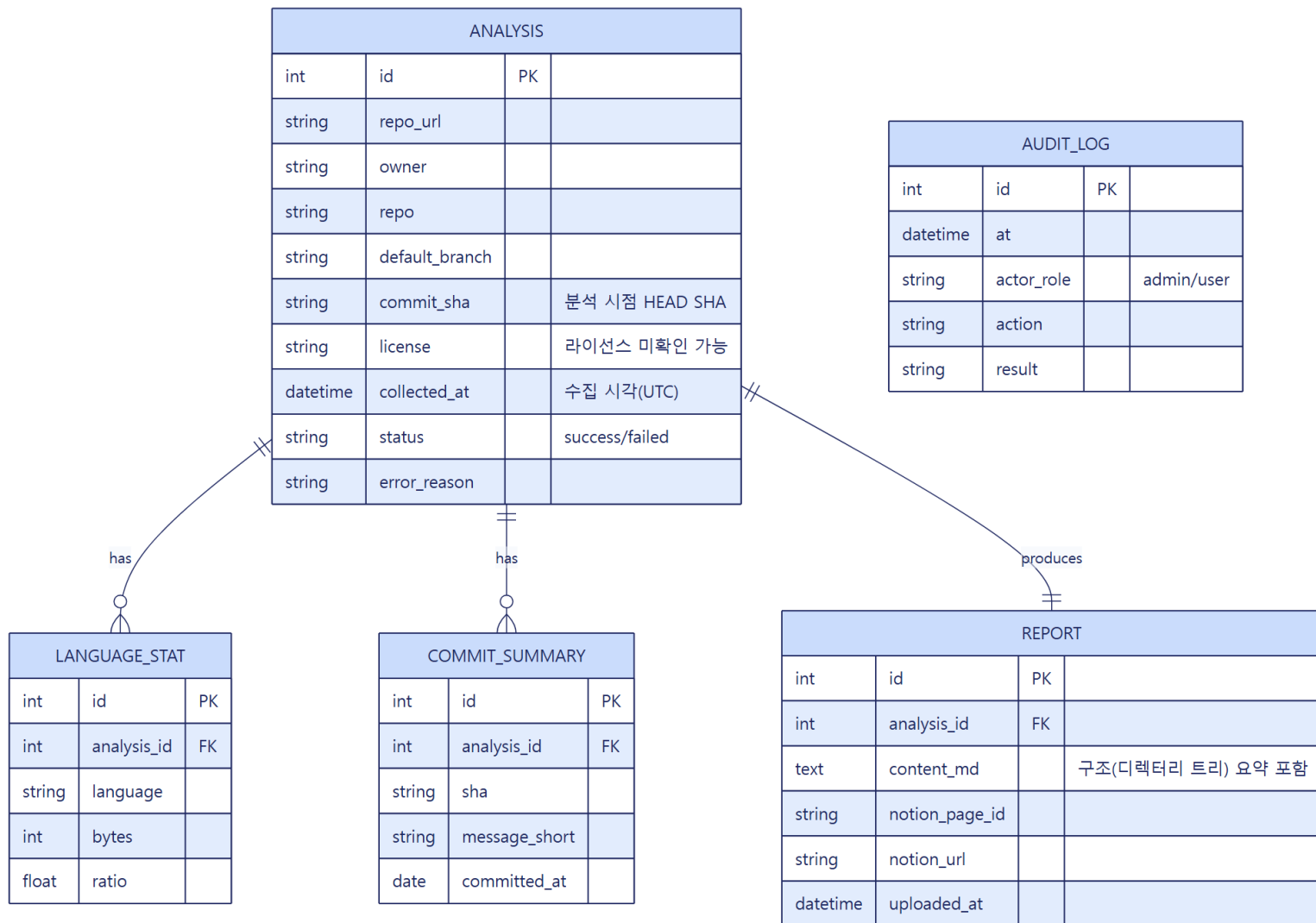


② 결과 미리보기



③ 이력 목록

3-3. 데이터 모델 (ERD)

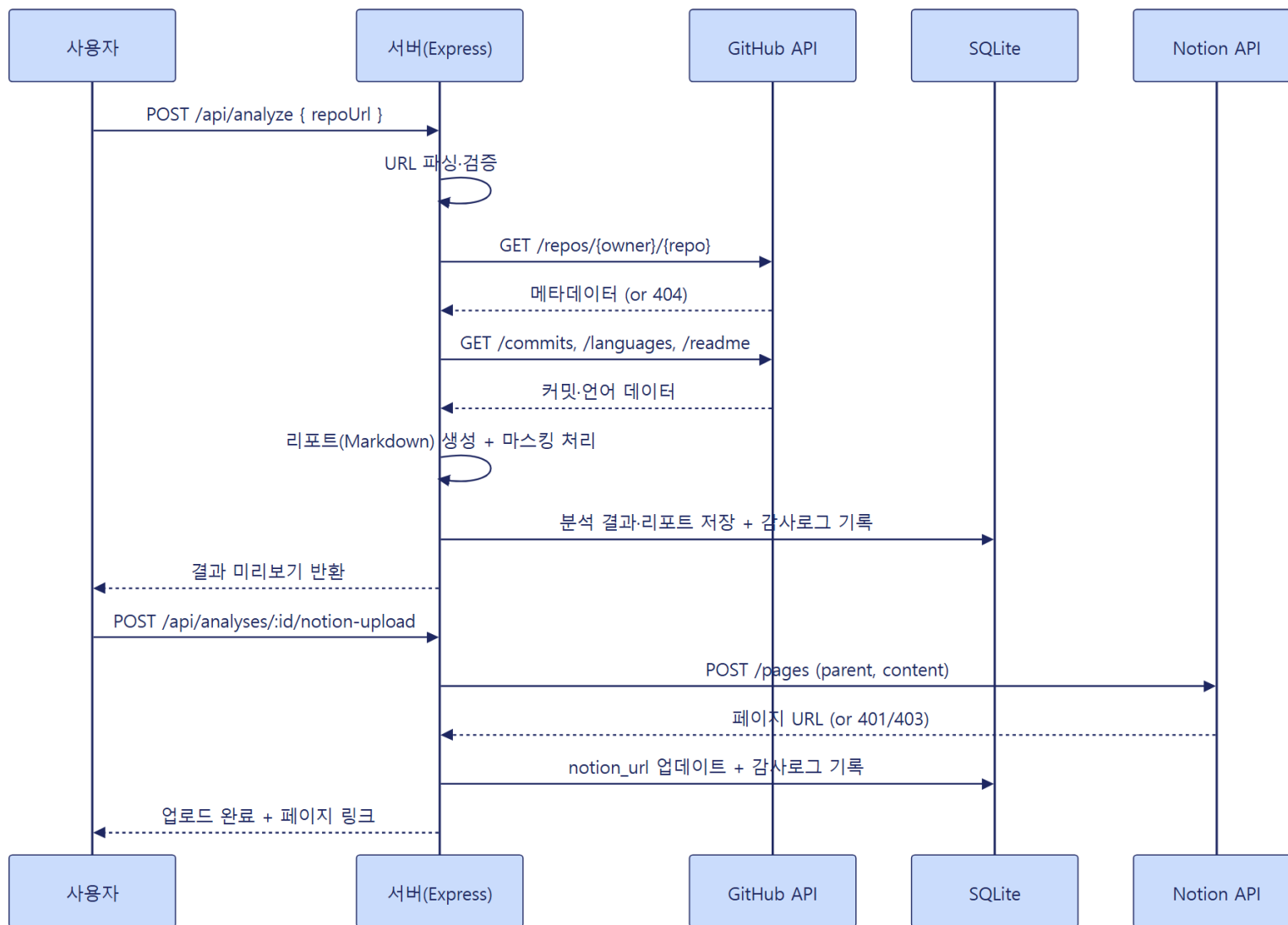


3-4. 백엔드 — API 명세

메서드	경로	설명	상태코드
POST	/api/analyze	레포 URL 분석 실행	200/400/404/429
GET	/api/analyses	분석 이력 목록 조회	200
GET	/api/analyses/:id	분석 상세 + 리포트 조회	200/404
POST	/api/analyses/:id/notion-upload	Notion 페이지 업로드(수동 트리거)	200/401/403/502
DELETE	/api/analyses/:id	이력 삭제 (관리자 전용)	200/403/404

- Notion 업로드는 분석 완료 시 자동 실행되지 않고, 사용자가 결과를 확인한 뒤 버튼을 클릭해야 실행되는 수동 트리거입니다.
- DELETE는 ?role=admin 이 아니면 서버가 실제로 403을 반환해 차단합니다 (단순 UI 숨김이 아닌 서버 측 접근 제어).

3-4. 백엔드 — 시퀀스 다이어그램



4. 테스트 결과



4. 테스트 결과 — 인수조건 6/6 커버리지

AC-ID	내용	커버 TC	결과
AC1	정상: 분석 실행 → 리포트 표시	7	0
AC2	정상: Notion 업로드 → 페이지 링크	2	0
AC3	정상: 이력 재조회	3	0
AC4	예외: 비공개/미존재/형식오류	4	0
AC5	예외: Rate Limit 초과	1	0
AC6	예외: Notion 토큰 미설정	2	0

19건 / 19건

단위 5·통합 7·E2E 7 전부 Pass

6 / 6

인수조건(AC) 전체 커버리지

- 실제 공개 레포 18건 이상으로 검증, Rate Limit·Notion 토큰 미설정 예외도 실제 재현



실제 Notion 업로드 성공 화면(레포 분석 예시)

5. 배포



5-1. 발생한 문제 (검증 환경·운영 환경에 실제로 배포하는 중)

- 1. 화면마다 결과가 다르게 보임** — 서버를 두 대로 늘려 운영했더니, 방금 처리한 결과를 다른 서버에서는 찾지 못했습니다. 각 서버가 데이터를 자기 컴퓨터에만 따로 저장하고 있었기 때문입니다.
- 2. 쓰지 않는 서비스까지 만들도록 설정되어 있었음** — 운영 환경 설정에는 이 앱이 전혀 쓰지 않는 대형 데이터베이스·저장소까지 새로 만들도록 되어 있었습니다. 처음 계획과 실제로 만든 결과물이 서로 맞지 않았던 것입니다.
- 3. 저장 실패를 '성공'으로 잘못 표시함** — 비밀번호 같은 민감한 값을 안전한 보관함에 저장하는 작업이 실제로는 실패했는데도, 자동화 프로그램은 성공했다고 알려줬습니다. 권한이 없어 생긴 문제를 프로그램이 확인하지 않고 그냥 넘어갔습니다.
- 4. 쓸 수 있는 자원 한도를 초과함** — 새 서버를 추가하려 했지만, 같은 지역에서 다른 환경들이 계정에 허용된 자원을 이미 다 써버려 더 만들 수 없었습니다. 코드를 고쳐서 해결할 수 있는 문제가 아니었습니다.
- 5. 이전에 고친 문제가 새 환경에는 반영되지 않음** — 배포를 관리해 주는 도구를 설치하는 중 설정 파일 용량이 허용 한도를 넘어 설치가 실패했습니다. 이전 환경에서 이미 겪고 고쳤던 방법이 새 운영 환경에는 그대로 적용돼 있지 않아 같은 문제가 반복된 것입니다.

- 직접 실행해 보지 않고 '설계상 문제없다'고 넘어갔다면 이 문제들 대부분을 발견하지 못했을 것입니다 — 실제로 돌려보는 것이 가장 확실한 확인 방법이었습니다.

5-2. 해결 방법

- 1. 서버 수를 잠깐 줄였다가 되돌림** — 서버를 한 대로 줄여 문제없이 동작하는 것을 확인한 뒤 다시 두 대로 되돌렸습니다. 이후 설정을 실수로 바뀌도 시스템이 10초 안에 스스로 원래대로 복구되는 것도 확인했습니다.
- 2. 쓰지 않는 서비스는 만들지 않도록 변경** — 실제로 쓰지 않는 대형 데이터베이스·저장소는 아예 만들지 않도록 설정을 바꿔, 불필요한 비용 없이 꼭 필요한 것만 준비했습니다.
- 3. 저장 권한을 추가하고 실패를 알려주도록 수정** — 보관함에 값을 저장할 권한을 계정에 추가로 부여하고, 저장이 실패하면 바로 알 수 있도록 프로그램을 고쳤습니다. 이후 필요한 값 3가지가 정상적으로 저장된 것을 확인했습니다.
- 4. 이미 있는 자원을 함께 쓰도록 전환** — 새 자원을 추가하는 대신 이미 있는 자원을 함께 쓰도록 바꿔 문제를 우회했습니다. 정식으로는 사용량 한도를 늘린 뒤 원래 방식으로 되돌리는 것이 바람직합니다.
- 5. 설정 적용 방법을 이전에 검증된 방식으로 통일** — 이전 환경에서 이미 검증된 방법으로 설정 적용 방식을 통일해 문제를 해결했습니다.

- 핵심 교훈: 데이터 저장 방식을 근본적으로 바꾸는 작업은 이번 범위에 포함하지 않았습니다 — 지금의 해결은 임시 조치라는 점을 문서에 남겨 다음에 이어서 개선하기로 했습니다.



6. 과정 회고

6. 과정 회고 — 현업 적용 & 다음 학습

반복 조사 업무 자동화 패턴 확산: 'URL 입력→자동 리포트→협업툴 업로드' — 건당 20~30분→5분(약 80% 절감), 출처(URL·시각) 자동 기록으로 감사 추적성 확보 (ROI 높음·난이도 낮음)

비밀 없는 접근 전면 적용: Key Vault + 워크로드 ID로 토큰 유출 가능성을 구조적으로 낮춤 (ROI 높음·난이도 중간)

배포 파이프라인 표준 템플릿화: 10~90단계 스크립트 + 80_verify 자동 검증을 팀 표준으로 (ROI 중간·난이도 중간)

다음 확장 포인트

[AI 기반 레포 주제·목적 분석] 현재는 커밋 메시지+구조 스냅샷 수준이지만, LLM으로 README·커밋 이력·구조를 종합 분석해 '이 프로젝트의 목적'과 개선 피드백을 리포트에 추가하는 기능으로 다음 단계 확장 가능

- 다중 레플리카에서도 정합성이 유지되는 데이터 계층(PostgreSQL/PVC 공유 스토리지)으로 전환
- GitHub/Notion 같은 서드파티 SaaS의 비밀 최소화 대안(OAuth App 등) 조사
- GitOps selfHeal의 '의도된 긴급 변경' vs '실수'를 구분하는 운영 절차 학습

GitHub Copilot 기본 기능과의 차별점

GitHub Copilot의 레포 개요 기능과 '분석' 아이디어는 겹치지만, myapp은 Notion 자동 업로드·이력 관리·권한 통제·배포 파이프라인까지 실제 운영 가능한 형태로 확장했다는 점이 다릅니다.



7. 시연

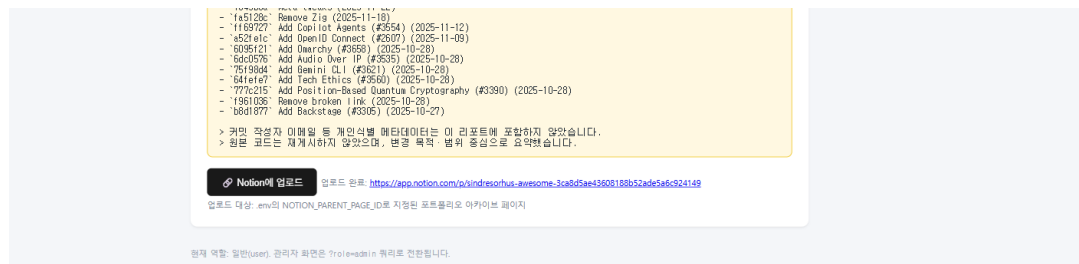
7. 시연 — 3분 임원 시연 시나리오

순서	클릭 / 화면	강조할 KPI
1 (30초)	메인 화면, 레포 URL 입력	URL 하나만 넣으면 끝
2 (60초)	분석 실행 → 결과 미리보기	20~30분 조사가 수십 초로 단축
3 (30초)	생성된 Markdown 리포트 확인	출처(URL·SHA·시각) 자동 기록
4 (30초)	Notion에 업로드 → 완료 링크	복사 없이 팀 공유 문서로 전환
5 (30초)	이력 목록 → 과거 이력 재조회	과거 분석도 언제든지 재조회

실행 화면 시연 영상

dev 환경(k3d, <http://localhost:8080>)에서 실제로 실행한 화면을 순서대로 재생하는 시연 자료입니다.

URL: <https://claude.ai/code/artifact/ae150cfb-eb10-4623-bc3f-d29b2a36d075>



dev 환경 실제 업로드 완료 화면

- 실측: dev run_all.ps1 8단계(10 전제조건 ~ 80 검증) 전부 Pass, stg(AKS+Argo CD) 재현 배포도 검증 완료

감사합니다
Thank you~!